

Track: Zero-Friction Production

The Mission: Build the "Missing Layer" between a beginner's code and a production URL.

The cloud is currently built for experts. We want you to build for the **Vibecoder**—someone who can prompt an AI or write a script, but feels paralyzed by AWS consoles, Dockerfiles, and IAM roles. Your goal is to replace fear with confidence through radical abstraction.

The Core Challenge

Create a tool, agent, or platform where a novice can ship a real, production-grade service in minutes without ever hearing the words "infrastructure" or "provisioning."

What You're Building

You have total freedom over the **Provider** (AWS, Modal, Fly.io, etc.) and the **Interface** (CLI, Chat, or Git), but the **Outcome** must meet these three pillars:

1. **Invisible Infrastructure:** The user shouldn't know (or care) which cloud is running their code. Abstract the complexity away entirely.
2. **Opinionated Safety:** Deployments must be "Safe-by-Default." You must implement at least three of these:
 - Automatic Secret Management
 - Auto-Restarts & Health Checks
 - Resource Limits (Cost Protection)
 - One-Click Rollbacks
 - Instant Versioning
3. **Human-Centric Feedback:** No cryptic stack traces. If a build fails, explain why in plain English and suggest a fix.

Scope & Scoring

We value **ambition** tempered by **usability**. While static sites are welcome, projects that tackle complex workflows will score higher.

High Complexity (High Score)	Standard Complexity
Long-running AI Agents / LLM Pipelines	Full-stack Web Apps
Stateful Services & Databases	Background Cron Jobs

Multi-service Orchestration	Simple Scripts / Bots
-----------------------------	-----------------------

Judging Criteria

- **The "Vibe" Check:** How quickly can a beginner go from "Hello World" to a live URL?
- **Abstraction Quality:** How elegantly did you hide the "scary" parts of the cloud?
- **Safety & Trust:** Does the system protect the user from expensive or insecure mistakes?
- **Innovation:** Did you use AI agents or new mental models to redefine "deployment"?

The Demo Requirement

To cross the finish line, you must show:

1. **A live deployment** of a real service.
2. **The "Safety Net"**—show us a mistake (like a crash or a leaked key) being caught or handled by your system.

Winning projects will make users say: *"I don't know how the cloud works—and I didn't need to."*